

JAN 2027 ① START DRAFTING JOURNAL ARTICLE FOR VALIDATION RESULTS →	FEB 2027	MAR 2027
APR 2027 ① MAKE CONFERENCE SLIDES ↓ ⑦ ⑮ M+C CONFERENCE ~ COLUMBUS, OH ↓ ⑲	MAY 2027 ① TARGET PRELIM EXAM! ⑦ START LUN RESEARCH	JUN 2027
JUL 2027	AUG 2027	SEP 2027 ① RESTART NSF FUNDING
OCT 2027	NOV 2027	DEC 2027

★ Ask Najel why "failure cannot occur in the control drum."

reactivity $\rightarrow \rho = \frac{\Delta K}{K} \left\{ \begin{array}{l} \% \text{ change} \\ \text{in criticality } K \end{array} \right.$

$\rightarrow$ study tried 8, 4, 2, & 1 drum cases

Control methods

PID: proportional integral derivative control

SMC: sliding model control

NMPC: nonlinear model predictive control

★ What does the $\leq 1100K$ fuel temp. constraint mean?

★ Should we try to validate all physical control algorithms? Or just best performing?

SPECIFICATION

Load-following simulation

ENVIRONMENT

Holos-Quad

HTGR Reactor

- nonlinear point kinetics
- thermal hydraulics
- poison + reactivity

Limitations:

- no spatial (only temporal) effects

* assumes uniform neutron distribution throughout the core

need a high fidelity model to test this on!
LBELAP or SAM

SYSTEM

CONTROL ALGORITHM
(PID, STC, NMPC, RL, MARL)

GOAL: validate this control algorithm!

Failure is defined as $\geq 1\%$ deviation from target power. (e.g.)

AND

available action is drum speeds (individual) as $\frac{d\theta}{dt}$, $dt = 1$ second (fixed)

★ do we force all CDs to operate symmetrically?

★ is the answer true for MARL & RL?

Fuel Temp.
Mod. Temp.
Coolant Temp.
Control Drum rotation
Control Drum Speed

Operating Variables

NEED TO CREATE SPECIFICATIONS FOR ALL OF THESE VARIABLES!

Core reactivity
Fuel temp.
Mod. temp.
Coolant temp.

Plant Variables

PER-STATE OBSERVATIONS

$P_t \equiv$ current measured power

$P_{t+1}^* \equiv$ the next desired power

$P_{t-1} \equiv$ the previous measured power

$\theta_t^1, \dots, \theta_t^8 \equiv$ the current control drum positions

* Noise was added in Led's paper to evaluate the impact.

↳ we will conduct a much more intensive version of this

→ 2 million timesteps!

* Why would you let the 8 control drums be perturbed separately if the model assumes a spatially uniform neutron distribution?

* MARL $\frac{1}{2}$ RL Outperform PID when noise is added

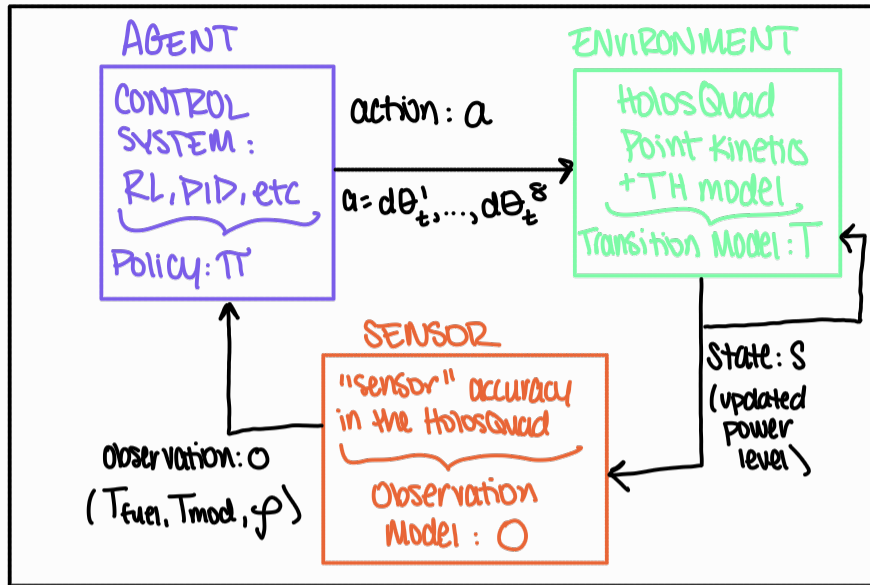
"Additional work...exploring the impact of biased measurements would be an important next step"



Motivation from Led's paper!

FALSIFICATION → "process of finding trajectories of a system that violate a specification"

Partially Observable Markov Decision Process



T } the transition model must satisfy the Markov assumption
 ↳ the next state only depends on the current state and the action
 this is true for our cases!
 $T(s'|s, a)$

def rollout(agent, environment, sensor, timesteps):

"trajectory" $S \leftarrow$ sampled from initial state distribution

↳ $\gamma = []$

for t in timesteps:

$O = \text{sensor}(S)$

$a = \text{agent}(O)$ ← "step"

$\gamma.append([S, O, a])$

$S' = \text{environment}(S, a)$

$S = S'$

return γ

Defining a Specification w/ Linear Temporal Logic

$$R \equiv p < 0.001$$

$$F \equiv 875K \leq T_{fuel} \leq 900K$$

$$M \equiv 800K \leq T_{mod} \leq 850K$$

DEFINE SUCCESS STATES

NATALY WILL NEED HELP DETERMINING THESE SPECIFICATIONS!

$$\Psi = \Box R \wedge \Box F \wedge \Box M$$

"always" "and"

$\Psi(\gamma) = \text{True}$
if $\gamma \in \Psi$ } Success

$\Psi(\gamma) = \text{False}$
if $\gamma \notin \Psi$ } Failure

check for reaching a failed state in a trajectory:

def specification(Ψ, γ):

for step in γ :

if $\Psi(\text{step})$:

continue

else:

return False

return True

RECALL

each "step" contains

[state, observation, action]

"Naive Monte Carlo"

METHOD #1: DIRECT SAMPLING

goal $\rightarrow$ find γ s.t. $\gamma \notin \Psi$

contains:
agent, environment, & sensor models

```
def direct_falsify(rollout,  $\Psi$ , system, timesteps, m):
```

number of samples
i.e., # of rollouts

```
     $\gamma_s = [\text{rollout}(\text{system}, \text{timesteps}) \text{ for } i \text{ in range}(m)]$ 
```

```
    return np.array[specification( $\Psi, \gamma$ ) for  $\gamma$  in  $\gamma_s$ ],  $\gamma_s$ 
```

$\rightarrow [\text{TRUE}, \text{TRUE}, \text{TRUE}, \dots, \text{FALSE}, \text{TRUE}, \text{TRUE}] , \gamma_s$

$\downarrow$

we can quantify the failure rate &
retrieve the failure mode from this

WE CAN DO SOME MATH TO UNDERSTAND A PROBLEM WITH THIS APPROACH:

$P_{\text{fail}} \equiv$ probability that a given rollout leads to a failure state

$P(k) \equiv$ probability that we sample the first failure on the k^{th} rollout

$$P(k) = (1 - P_{\text{fail}})^{k-1} P_{\text{fail}}$$

geometric distribution $\rightarrow$

$$\therefore \text{the } E(k) = 1/P_{\text{fail}}$$

$$\therefore \text{if } P_{\text{fail}} = 1 \times 10^{-9},$$

we need 1×10^9
(1 Billion) trials to find a
single failure

How can we find these failures more efficiently?

↳ By controlling sources of RANDOMNESS $\Rightarrow$ ADD DISTURBANCES

observation model $\rightarrow o \sim O(\cdot|s)$ } stochastic + deterministic components wrapped together

$$o = \underbrace{O(s, x_o)}_{\text{deterministic component}}, \quad \underbrace{x_o \sim D_o(\cdot|s)}_{\text{stochastic component}}$$

$D_o(\cdot|s) \equiv$ Disturbance Distribution

e.g., Gaussian noise model

$$o \sim \mathcal{N}(\cdot|s, \Sigma)$$
 } observation data are normally distributed around the state s

$$o = \underbrace{s + x_o}_{\text{deterministic function}}, \quad \underbrace{x_o \sim \mathcal{N}(\cdot|0, \Sigma)}_{\text{center the disturbance distribution at 0}}$$

REPEAT FOR AGENT & ENVIRONMENT

agent $\rightarrow$ $\overset{\text{action}}{\downarrow} a = \pi(\cdot|o) = \pi(o, x_a), \quad x_a \sim D_a(\cdot|o)$
 $\uparrow$ policy model
 $\uparrow$ observation

environment $\rightarrow s' = T(\cdot|s, a) = T(s, a, x_s), \quad x_s \sim D_s(\cdot|s, a)$
 $\uparrow$ future state
 $\uparrow$ transition model
 $\uparrow$ current action state

If a system component * is deterministic, set its disturbance distribution to 0

Since this can get messy, wrap all 3 together $\rightarrow$

$$X = (x_s, x_o, x_a) \quad \& \quad D = (D_s, D_o, D_a)$$

Method #2 → Fuzzing

*this has been approximately done in Leo's paper

WE WILL
⇒ START HERE
FOR M&C TO
FORMALIZE &
EXPAND LEO'S
VALIDATION STUDY.

(exploiting the disturbance to make failures more likely)

OLD

```
def rollout(agent, environment, sensor, timesteps):  
    "trajectory" s ← sampled from initial state distribution  
    γ = [ ]  
    for t in timesteps:  
        o = sensor(s)  
        a = agent(o)  
        s' = environment(s, a) ] "step"  
        γ.append([s, o, a])  
        s = s'  
    return γ
```

class System:

```
def __init__(self, sensor, agent, environment):  
    self.sensor = sensor  
    self.agent = agent  
    self.env = environment
```

class Disturbance:

```
def __init__(self, xo, xa, xs):  
    self.xo = xo ← sensor  
    self.xa = xa ← agent  
    self.xs = xs ← environment
```

class DisturbanceDistribution:

```
def __init__(self, Do, Da, Ds):  
    self.Do = Do ← sensor  
    self.Da = Da ← agent  
    self.Ds = Ds ← environment
```

```

def step(sys, s, D):
    x0 = D.D0(s).sample()
    o = sys.sensor(s, x0)
    xa = D.Da(o).sample()
    a = sys.agent(o, xa)
    xs = D.Ds(s, a).sample()
    s' = sys.env(s, a, xs)
    x = Disturbance(x0, xa, xs)
    return [o, a, s', x]

```

PyTorch: .sample()
 Seipy.stats: .rvs()

get observation

get action

get state

← wrap disturbance as object

the "nominal" part of this just means it does not depend on time

```

class NominalTrajectoryDistribution:

```

```

    def __init__(self, initial_state_distribution, DisturbanceDistribution, depth):
        self.isd = initial_state_distribution
        self.D = DisturbanceDistribution
        self.d = depth

```

NEW

```

def rollout(system, ntd):
    s = ntd.isd.sample()
    γ = [ ]
    for t in ntd.d:
        o, a, s', x = step(system, s, ntd.D)
        γ.append([s, o, a, x])
        s = s'
    return γ

```

sample initial state

"depth"

the Disturbance Distribution

* since we're using the NTD, we do not need t, but time dependent distributions will

FUZZED ROLLOUT

```
def rollout(system, fuzzy_distribution):  
    s = fd.isd.sample() ← sample initial state  
     $\gamma = [ ]$   
    for t in fd.d: ← "depth"  
        o, a, s', x = step(system, s, fd.D(t)) ← the Disturbance Distribution  
         $\gamma.append[s, o, a, x]$  ← if t is required!  
        s = s'  
    return  $\gamma$ 
```

* from this, we can repeat the falsify algorithm and get a list of trajectories and their failure masks } failure rate w/ disturbed distribution + failed trajectories

What if we wanted to search for the trajectories that fail?

Method #3 — Falsification through Optimization

① rewrite our stochastic (fuzzed) rollout to be deterministic

disturbance object $\rightarrow x = (x_0, x_a, x_s)$

```
def step(sys, s, x):  
    o = sys.sensor(s, x.x0)  
    a = sys.agent(o, x.xa)  
    s' = sys.env(s, a, x.xs)  
    return [o, a, s']
```

different b/c we do not sample the disturbances, instead we give them to the function directly through x

disturbance vector: $\vec{x} = (x_1, \dots, x_d)$

```
def rollout(sys, s,  $\vec{x}$ ):  
    d = len( $\vec{x}$ )  
     $\gamma = []$   
    for t in d:  
         $x = \vec{x}[t]$   
        o, a, s' = step(sys, s, x)  
         $\gamma.append([o, a, s, x])$   
         $s = s'$   
    return  $\gamma$ 
```

* More info on objective functions & optimization algorithms available if we want to pursue this route!

Objective

find initial state s & disturbance trajectory $\vec{x}$ that yield a γ s.t. $\gamma \notin \Psi$

minimize $f(\gamma)$ subject to $\gamma = \text{Rollout}(s, \vec{x})$
 $s, \vec{x}$

What if we wanted to make the search process even more efficient by considering partial trajectories?

Method #4 — Falsification through Planning

- ① one option is to break up the optimization method by optimizing over a sequence of "sub-trajectories"

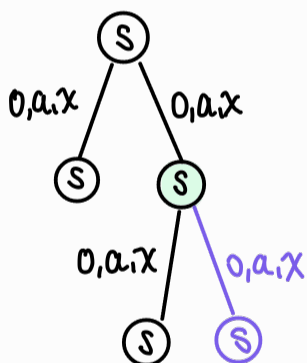
↳ this method does not work well for black-box simulators, so we won't consider details of this for now

② Tree Search

nodes $\equiv$ states

edges $\equiv$ transition between states caused by a particular disturbance

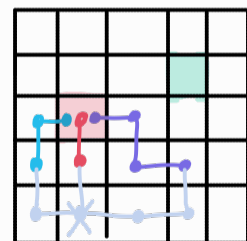
Generic Tree Search



① select a promising node

② extend the tree

Grid world



Rapidly Expanding Random Trees (RRT)

- ① randomly sample a goal state from the state space

↳ e.g., the red highlighted square (not a success path—green)

- ② compute an objective function for each node in the current tree based on the sampled state

↳ e.g., shortest distance to goal state from child nodes

- ③ select node

- ④ extend from node using disturbance from nominal disturbance distribution

Monte Carlo Tree Search

* THIS WILL BE THE FIRST NOVEL PART OF THE RESEARCH!

* SAME STRUCTURE AS RRT BUT INCLUDES BACKPROPAGATION OF SCORES GENERATED BY EACH ROLLOUT (INCLUDES RANDOMLY SAMPLED "STEPS TO END STATE" → THE MONTE CARLO PART) & AN OBJECTIVE FUNCTION TO BALANCE EXPLORATION & EXPLOITATION.

⇒ GOAL: FIND & ANALYZE FAILURE TRAJECTORIES